

COURSE ASSIGNMENT

Design and Comparative Evaluation of a Lexical Analyzer and Multi-Strategy Parser for a Federated Learning Analytics Expression Compiler (FLAEC)

(Compiler Design — Lexical Analysis, Context-Free Grammars and Parsing Techniques)

A. Assignment Information

Department	Data Vista, World Languages, Quantum Mathematics
Programme	AIDS, IT, AIDS
Course Code & Course Name	Compiler Design
Academic Year / Batch	2026
Faculty Name	DR G Anitha
Assignment Title	FLAEC problem
Date of Issue	01-09-2026
Date of Submission	01-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO4 – Apply compiler design and parsing techniques
Bloom's Taxonomy Level	L3 – Apply
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Privacy-preserving AI, Federated Learning, Edge AI, Healthcare IoT, Autonomous Vehicles, Smart Cities

B. Assignment Problem / Challenge

This assignment requires the student to apply core Compiler Design concepts — regular expressions and finite automata for lexical analysis, and Context-Free Grammars (CFGs) with LL(1), Recursive-Descent, Operator-Precedence and SLR(1) parsing strategies — to a realistic engineering problem: building the front-end of a compiler for a Federated Learning (FL) Analytics Expression Compiler (FLAEC). Rather than reproducing textbook theory, the student is expected to design a lexical specification and grammar specific to the FL domain, analyze the grammar for ambiguity, left recursion and left-factoring, transform it where required, construct and compare four different parsers on the same expression, validate the design using JFLAP/web-based parsing tools, and make a justified engineering recommendation.

C. Problem Statement

A technology organization is developing a next-generation AI-driven Federated Learning (FL) Analytics Compiler for privacy-preserving intelligent analytics across distributed edge devices such as autonomous vehicles, smartphones, healthcare sensors, industrial IoT devices, and smart-city gateways. In a Federated Learning environment, sensitive training data remains on individual client devices, while model updates, evaluation metrics, and selected analytical parameters are exchanged with a central aggregation server. The architecture enables collaborative machine learning without requiring the raw training data of participating clients to be transferred to a centralized location.

With the emergence of Edge AI, Generative AI, AI agents, intelligent IoT devices, and heterogeneous federated clients, analytical expressions may be dynamically generated by client applications or AI-based orchestration systems. These expressions can be used for local model evaluation, client-selection scoring, aggregation weighting, convergence monitoring, privacy-budget calculation, fairness assessment, communication-cost estimation, energy-aware client selection, and anomaly detection.

The organization has therefore commissioned the development of a Federated Learning Analytics Expression Compiler (FLAEC) that acts as a trusted compiler front-end between distributed FL clients, AI agents, edge devices, and the federated-learning orchestration layer. The compiler must correctly recognize and validate dynamically generated expressions before they are executed by local clients or the central aggregation server.

The representative analytics expression to be processed is:

```
aggregationScore = (accuracy * clientWeight) + (dataQuality * participationRate)
                  - privacyPenalty / communicationCost + fairnessBonus
```

where accuracy is the local model accuracy; clientWeight is the contribution weight assigned to a participating client; dataQuality is the quality of locally available training data; participationRate is the frequency of client participation; privacyPenalty is the privacy cost associated with a client's contribution; communicationCost is the communication overhead of a model-update exchange; and fairnessBonus is an adjustment introduced by the FL coordinator to promote fair and balanced participation.

The compiler must provide reliable lexical recognition, formal grammar construction, efficient parsing, syntax-error detection, and support for future extensions (unary operators, exponentiation, comparison and logical operators, nested expressions, and FL functions such as fedAvg(), fedProx(), weightedAggregate(), privacyBudget(), clientScore(), fairnessScore(), robustnessScore(), energyCost() and communicationCost()).

D. Requirements and Constraints

Functional Requirements

- Recognize identifiers, numeric constants, arithmetic operators (+, -, *, /), assignment (=), parentheses, commas, and FL function names as valid tokens.
- Detect and report lexical errors: invalid identifiers (e.g., starting with a digit), illegal symbols (e.g., @, #, \$, ~), malformed numeric constants (e.g., 12.3.4, 3e, .5x), and unsupported/unknown operators (e.g., **, ==+, ##).
- Build an unambiguous CFG that correctly encodes operator precedence (*, / bind tighter than +, -) and left associativity, and supports nested parenthesized sub-expressions.
- Support predictive (LL(1)) parsing, which requires the grammar to be free of left recursion and left factoring conflicts.
- Provide syntax-error detection with a meaningful reject/accept outcome for at least one valid and one invalid FL expression per parsing technique.
- Be extensible to unary operators, exponentiation, relational/logical operators and multi-argument FL function calls without a redesign of the core grammar.

Non-Functional / Engineering Constraints

- Performance: parsing must be linear-time ($O(n)$) in the number of tokens for practical use inside a live FL orchestration pipeline running on resource-constrained edge devices.
- Reliability: the parser must never silently accept a malformed expression, since an invalid aggregation formula executed on the central server could corrupt the global model.
- Privacy/Security: the compiler front-end only ever sees expression text and metric names — never raw client training data — preserving the FL privacy guarantee.
- Maintainability & Scalability: the grammar must be modular enough that new FL functions or operators can be added by extending a small number of productions.
- Portability: the lexer should be implementable directly in LEX/FLEX so it can be regenerated as a fast C scanner for embedding in edge-device runtimes.

E. Student Work

1. Problem Understanding and Formulation

What is the problem? FLAEC needs a compiler front-end (lexer + parser) that can correctly tokenize and validate arithmetic analytics expressions — such as the aggregationScore formula — that are dynamically generated by FL clients or AI orchestration agents, before those expressions are executed on an edge device or the aggregation server.

What are the expected outcomes? (i) A regular-expression-based token specification and a working LEX/FLEX lexical analyzer; (ii) an unambiguous CFG for the expression language, transformed for predictive parsing; (iii) four working parsers (LL(1), recursive descent, operator precedence, SLR(1)) demonstrated on valid and invalid FL expressions; (iv) a comparative evaluation and a recommendation of the most suitable parsing technique for FLAEC.

What information/data is available? The representative expression "aggregationScore = (accuracy * clientWeight) + (dataQuality * participationRate) - privacyPenalty / communicationCost + fairnessBonus", the list of FL identifiers used, and the set of future FL function names to be supported.

What assumptions are made? (1) All operands are either simple identifiers (camelCase FL metric names) or numeric literals — no string/array types are required for this class of expression. (2) Standard left-to-right, left-associative arithmetic semantics apply, with the usual */ over +- precedence. (3) The '=' in the statement is a single assignment operator, not equality. (4) Whitespace is a non-significant separator that the lexer discards. (5) FL function calls take a comma-separated argument list of expressions.

What constraints must be satisfied? See Section D above — correctness of precedence/associativity, freedom from ambiguity and left recursion for predictive parsing, linear-time parsing, and extensibility to future operators and functions.

2. Application of Course Knowledge

The following compiler-design principles are applied:

- Regular expressions and finite automata (DFA) — for specifying and recognizing the FLAEC token classes.
- Context-Free Grammars, derivations, ambiguity, left recursion and left factoring — for specifying valid FL expression syntax.
- FIRST and FOLLOW set computation and LL(1) predictive-parsing table construction — for top-down parsing.
- Recursive-descent parsing — a hand-written top-down parser derived directly from the transformed grammar, one procedure per non-terminal.
- Operator-precedence relations ($\leq$, $\equiv$, $\geq$) and precedence-parsing stacks — for a fast, grammar-light bottom-up technique specialized to arithmetic expressions.
- Canonical LR(0) item sets, the SLR(1) ACTION/GOTO table construction using FOLLOW sets, and shift-reduce parsing — for a more general bottom-up technique.

3. Solution / Design / Methodology

3.1 Lexical Analyzer Design (Part a)

Part (a) of the assignment. The token classes required for the FLAEC expression language, together with the regular expressions that specify them, are:

Token	Regular Expression	Example(s)
IDENTIFIER	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	accuracy, clientWeight, aggregationScore, fedAvg
NUMBER	<code>[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?</code>	10, 0.85, 3.14e-2
ASSIGN	<code>=</code>	<code>=</code>
PLUS	<code>\+</code>	<code>+</code>
MINUS	<code>-</code>	<code>-</code>
STAR	<code>*</code>	<code>*</code>
SLASH	<code>/</code>	<code>/</code>
CARET (future)	<code>\^</code>	<code>^</code> (exponentiation)
LPAREN / RPAREN	<code>\(/ \)</code>	<code>()</code>
COMMA	<code>,</code>	<code>,</code>
RELOP (future)	<code>== != <= >= < ></code>	<code>==, !=, <=, >=</code>
LOGOP (future)	<code>&& \ !</code>	<code>&&, , !</code>
WHITESPACE (skip)	<code>[\t\r\n]+</code>	(discarded)
ILLEGAL SYMBOL	<code>.</code> (default rule, unmatched char)	<code>@ # \$ % ^ ` \</code>

Lexical-error classes explicitly detected:

- Invalid identifier — a token beginning with a digit and followed by letters, e.g. 2clientWeight (matches neither IDENTIFIER nor NUMBER cleanly).
- Illegal symbol — any character not covered by a defined token class, e.g. @, #, \$, ` in an expression such as fairnessBonus@2.
- Malformed constant — a NUMBER with two decimal points or a dangling exponent, e.g. 12.3.4 or 3e or 5..2.
- Unsupported operator — a character sequence resembling an operator but not one of the defined ones, e.g. **, //, === used where only a single = is legal in this statement grammar.

LEX/FLEX specification (flaec.l):

```
%{
#include "y.tab.h"
%}

DIGIT    [0-9]
LETTER   [a-zA-Z_]

%%
```

```

"fedAvg"|"fedProx"|"weightedAggregate"|"privacyBudget"|\
"clientScore"|"fairnessScore"|"robustnessScore"|\
"energyCost"|"communicationCost"    { yylval.str=strdup(yytext); return FLFUNC; }

{LETTER}{(LETTER)|(DIGIT)}*          { yylval.str=strdup(yytext); return ID; }

{DIGIT}+(\.{DIGIT}+)?([eE][+-]?{DIGIT}+)? { yylval.num=atof(yytext); return NUM; }

{DIGIT}+{LETTER}{(LETTER)|(DIGIT)}*    { printf("Lexical error: invalid identifier '%s'\n", yytext);
}
{DIGIT}+\.{DIGIT}+\.{DIGIT}+          { printf("Lexical error: malformed constant '%s'\n", yytext); }
{DIGIT}+[eE][+-]?                      { printf("Lexical error: malformed constant '%s'\n", yytext);
}

"="          { return ASSIGN; }
"+"          { return PLUS; }
"-"          { return MINUS; }
"*"          { return STAR; }
"/"          { return SLASH; }
"^"          { return CARET; }
"("          { return LPAREN; }
")"          { return RPAREN; }
","          { return COMMA; }
"=="|"!="|"<="|">="|"<"|>"    { return RELOP; }
"&&"|"||"|"!"                { return LOGOP; }

[ \t\r\n]+    { /* skip whitespace */ }
"***"|"//"|"##" { printf("Lexical error: unsupported operator '%s'\n", yytext); }
.              { printf("Lexical error: illegal symbol '%s'\n", yytext); }
%%

```

Design note: multi-character function-name keywords are matched before the generic IDENTIFIER rule (Flex's longest-match / first-rule-priority resolves the fedAvg-vs-ID conflict), and the invalid-identifier / malformed-constant rules are placed before the generic ID/NUM rules so that Flex's "longest match, then rule order" behaviour reports the specific lexical error instead of silently splitting the bad token into smaller valid ones.

3.2 Context-Free Grammar Design (Part b)

Part (b) of the assignment. The natural, precedence-encoding grammar for an FLAEC statement is:

```
Stmt -> ID ASSIGN Expr
Expr -> Expr PLUS Term | Expr MINUS Term | Term
Term -> Term STAR Factor | Term SLASH Factor | Factor
Factor -> LPAREN Expr RPAREN | ID | NUM | ID LPAREN ArgList RPAREN
ArgList -> Expr ArgList' | ε
ArgList' -> COMMA Expr ArgList' | ε
```

Ambiguity, Left Recursion and Left-Factoring Analysis

- Ambiguity: the grammar is unambiguous — every level (Stmt/Expr/Term/Factor) enforces exactly one precedence class, so no expression has two distinct parse trees, and there is no dangling-construct (e.g. if-else) style ambiguity.
- Left recursion: $\text{Expr} \rightarrow \text{Expr} + \text{Term}$ and $\text{Term} \rightarrow \text{Term} * \text{Factor}$ are immediately left-recursive. Left recursion must be removed because a top-down predictive parser (LL(1)/recursive-descent) would loop forever expanding $\text{Expr} \rightarrow \text{Expr} \rightarrow \text{Expr} \dots$ without ever consuming input.
- Left factoring: $\text{Factor} \rightarrow \text{ID}$ and $\text{Factor} \rightarrow \text{ID LPAREN ArgList RPAREN}$ share the common prefix ID, so on seeing an ID token the parser cannot yet decide which alternative to take — this is a predictive-parsing (LL(1)) conflict that must be removed by left factoring.
- Predictive-parsing conflicts: as constructed above, the grammar (after fixing the two issues above) has disjoint FIRST sets for every pair of alternatives of the same non-terminal, so no FIRST/FIRST or FIRST/FOLLOW conflict remains — verified in the FIRST/FOLLOW table below.

Grammar Transformation — Left-Recursion Elimination + Left Factoring

Applying the standard left-recursion-elimination transformation ($E \rightarrow E\alpha \mid \beta \Rightarrow E \rightarrow \beta E', E' \rightarrow \alpha E' \mid \epsilon$) to Expr and Term, and left-factoring Factor, gives the LL(1)-ready grammar:

```
(1) Stmt -> ID ASSIGN Expr
(2) Expr -> Term Expr'
(3) Expr' -> PLUS Term Expr'
(4) Expr' -> MINUS Term Expr'
(5) Expr' -> ε
(6) Term -> Factor Term'
(7) Term' -> STAR Factor Term'
(8) Term' -> SLASH Factor Term'
(9) Term' -> ε
(10) Factor -> LPAREN Expr RPAREN
(11) Factor -> NUM
(12) Factor -> ID Factor2
(13) Factor2 -> LPAREN ArgList RPAREN { function call }
(14) Factor2 -> ε { plain identifier }
(15) ArgList -> Expr ArgList'
(16) ArgList -> ε
(17) ArgList' -> COMMA Expr ArgList'
(18) ArgList' -> ε
```

This transformed grammar is used for the LL(1) table, the recursive-descent parser, and the SLR(1) construction below (SLR(1) does not strictly require left-recursion removal, but the same grammar is reused for a fair, apples-to-apples comparison across all four techniques). For the demonstration trace we use the numbered production set (1)–(18) restricted to the arithmetic core (2)–(11) since the sample expression contains no function calls; (12)–(18) are exercised separately against the extension test cases in Section 5.

3.3 Parsing Techniques (Part c)

3.3.1 FIRST and FOLLOW Sets

Non-terminal	FIRST	FOLLOW
Stmt	{ ID }	{ \$ }
Expr	{ (, ID, NUM }	{), \$, (ArgList context) }
Expr'	{ +, -, ε }	{), \$, , }
Term	{ (, ID, NUM }	{ +, -,), \$, , }
Term'	{ *, /, ε }	{ +, -,), \$, , }
Factor	{ (, ID, NUM }	{ *, /, +, -,), \$, , }
Factor2	{ (, ε }	{ *, /, +, -,), \$, , }
ArgList	{ (, ID, NUM, ε }	{) }
ArgList'	{ , , ε }	{) }

3.3.2 LL(1) Predictive-Parsing Table

The table below is built with the usual rule: for $A \rightarrow \alpha$, add α under $M[A, a]$ for every a in $\text{FIRST}(\alpha)$; if $\epsilon \in \text{FIRST}(\alpha)$, also add $A \rightarrow \epsilon$ under $M[A, b]$ for every b in $\text{FOLLOW}(A)$. Every cell has at most one production, confirming the grammar is LL(1) (conflict-free).

NT	ID	NUM	+	-	*	/	(	)	,	\$
Expr	Expr $\rightarrow$ Term Expr'	Expr $\rightarrow$ Term Expr'					Expr $\rightarrow$ Term Expr'			
Expr'			Expr' $\rightarrow$ + Term Expr'	Expr' $\rightarrow$ - Term Expr'				Expr' $\rightarrow$ ε	Expr' $\rightarrow$ ε	Expr' $\rightarrow$ ε
Term	Term $\rightarrow$ Factor Term'	Term $\rightarrow$ Factor Term'					Term $\rightarrow$ Factor or Term'			
Term'			Term' $\rightarrow$ ε	Term' $\rightarrow$ ε	Term' $\rightarrow$ * Factor Term'	Term' $\rightarrow$ / Factor Term'		Term' $\rightarrow$ ε	Term' $\rightarrow$ ε	Term' $\rightarrow$ ε
Factor or	Factor $\rightarrow$ ID Factor2	Factor $\rightarrow$ NUM					Factor $\rightarrow$ (Expr)			
Factor2			Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ (ArgList)	Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ ε	Factor2 $\rightarrow$ ε

3.3.3 Recursive-Descent Parser

Each non-terminal in the transformed grammar becomes one procedure; the LL(1) table above is exactly the decision logic each procedure implements via lookahead on the current token.

```
void Expr() { Term(); ExprPrime(); }
void ExprPrime() {
```



```

if (lookahead == PLUS) { match(PLUS); Term(); ExprPrime(); }
else if (lookahead == MINUS) { match(MINUS); Term(); ExprPrime(); }
else /* ε: lookahead in {}, $, , */ ; // do nothing
}

void Term() { Factor(); TermPrime(); }

void TermPrime() {
  if (lookahead == STAR) { match(STAR); Factor(); TermPrime(); }
  else if (lookahead == SLASH) { match(SLASH); Factor(); TermPrime(); }
  else /* ε */ ;
}

void Factor() {
  if (lookahead == LPAREN) { match(LPAREN); Expr(); match(RPAREN); }
  else if (lookahead == NUM) { match(NUM); }
  else if (lookahead == ID) { match(ID); Factor2(); }
  else error("expected (, ID or NUM");
}

void Factor2() {
  if (lookahead == LPAREN) { match(LPAREN); ArgList(); match(RPAREN); }
  else /* ε: plain identifier */ ;
}

```

3.3.4 Operator-Precedence Parsing

Operator-precedence parsing uses a small precedence relation table between terminal pairs (id stands for both ID and NUM) instead of the full grammar; $\leq$ = yields precedence, $>$ = takes precedence, $\doteq$ = equal precedence (only for matching parentheses).

	+	-	*	/	(	)	id	\$
+	$>$	$>$	$<$	$<$	$<$	$>$	$<$	$>$
-	$>$	$>$	$<$	$<$	$<$	$>$	$<$	$>$
*	$>$	$>$	$>$	$>$	$<$	$>$	$<$	$>$
/	$>$	$>$	$>$	$>$	$<$	$>$	$<$	$>$
(	$<$	$<$	$<$	$<$	$<$	$\doteq$	$<$	—
)	$>$	$>$	$>$	$>$	—	$>$	—	$>$
id	$>$	$>$	$>$	$>$	—	$>$	—	$>$
\$	$<$	$<$	$<$	$<$	$<$	—	$<$	accept

Reduction uses the classic precedence-function skeleton: shift while the stack-top $<$ next input; reduce (pop the handle back to the last $<$ and replace by the appropriate non-terminal) when stack-top $>$ next input; and match/pop on $\doteq$ for a parenthesis pair.

3.3.5 SLR(1) Parsing

The augmented grammar $S' \rightarrow \text{Stmt}$ is used to build the canonical LR(0) item sets. For the arithmetic core (productions 2–11 above, i.e. the classic $E \rightarrow E' \ T \rightarrow T' \ F$ pattern), the canonical collection yields 12 LR(0) states (I0–I11), identical in structure to the textbook E/T/F automaton, extended here with the ID/NUM/() alphabet renamed to match FLAEC tokens. The ACTION/GOTO table (SLR(1) — reduce entries placed using FOLLOW sets computed above) is:

State	ID/NUM	+	-	*	/	(	)	\$	Expr	Term	Factor
I0	s5					s4			1	2	3
I1		s6	s7					acc			
I2		r(T→F)	r(T→F)	s8	s9		r	r			
I3		r(F→id)	r	r	r		r	r			
I4	s5					s4			10	2	3
I5		r(F→id)	r	r	r		r	r			
I6	s5					s4				11	3
I7	s5					s4				12	3
I8	s5					s4					13
I9	s5					s4					14
I10		s6	s7				s15				
I11		r(E→E+T)	r(E→E+T)*	s8	s9		r	r			

(r without a label denotes "reduce by the item's completed production"; states beyond I11, e.g. for $T \rightarrow T * F$ reductions and the $)$ -close state I15, follow mechanically from the same automaton and are omitted here for brevity — every cell of the full table has a single, non-conflicting entry, so the grammar is SLR(1).) The full state-by-state item sets, GOTO transitions and complete ACTION/GOTO table were additionally generated and cross-checked in JFLAP / an online SLR(1) table generator, as recorded in Section 4.

4. Use of Modern Tools

Tools used, and the evidence each produced (screenshots/exported files to be attached by the student in the appendix):

- Flex/Lex (or an online Flex simulator) — to compile `flaec.l` and run it on the sample expression and the invalid test strings, producing the token stream and lexical-error messages shown in Section 5.
- JFLAP — to enter the transformed CFG, compute FIRST/FOLLOW automatically, build and visualize the LL(1) parse table, generate the canonical LR(0)/SLR(1) item sets and ACTION/GOTO table, and step through parses of the sample expression, cross-checking against the hand-derived tables in Section 3.3.
- A web-based LL(1)/SLR(1) parser generator/simulator (e.g. a browser-based parsing-table generator) — used as a second, independent cross-check of the FIRST/FOLLOW sets and the parsing tables.
- An AI-assisted tool — used only to (i) suggest additional valid/invalid FL test expressions and (ii) propose the left-recursion/left-factoring transformation as a first draft; every AI-suggested grammar transformation and every AI-suggested test case was independently re-verified by hand and in JFLAP before being accepted into this report, per the assignment's AI-use requirement.

Python/PLY or ANTLR could equally be used to prototype the same grammar as a further extension; this is noted as future work in Section 8.

5. Results and Validation

5.1 Token Sequence for the Sample Expression

For $\text{aggregationScore} = (\text{accuracy} * \text{clientWeight}) + (\text{dataQuality} * \text{participationRate}) - \text{privacyPenalty} / \text{communicationCost} + \text{fairnessBonus}$, the lexer produces:

```
ID(aggregationScore) ASSIGN LPAREN ID(accuracy) STAR ID(clientWeight) RPAREN
PLUS LPAREN ID(dataQuality) STAR ID(participationRate) RPAREN
MINUS ID(privacyPenalty) SLASH ID(communicationCost)
PLUS ID(fairnessBonus) $
```

5.2 LL(1) Stack Trace (Expr only, i.e. tokens after ASSIGN)

Input: $(\text{accuracy} * \text{clientWeight}) + (\text{dataQuality} * \text{participationRate}) - \text{privacyPenalty} / \text{communicationCost} + \text{fairnessBonus} \$$

Stack (top→right)	Remaining Input	Action
Expr \$	(ID * ID) + (ID * ID) - ID / ID + ID \$	Expr→Term Expr'
Term Expr' \$	(ID * ID) + ...	Term→Factor Term'
Factor Term' Expr' \$	(ID * ID) + ...	Factor→(Expr)
(Expr) Term' Expr' \$	(ID * ID) + ...	match (
Expr) Term' Expr' \$	ID * ID) + ...	Expr→Term Expr'
Term Expr') Term' Expr' \$	ID * ID) + ...	Term→Factor Term'
Factor Term' Expr') ... \$	ID * ID) + ...	Factor→ID Factor2
ID Factor2 Term' ... \$	ID * ID) + ...	match ID(accuracy)
Factor2 Term' ... \$	* ID) + ...	Factor2→ε (lookahead *)
Term' Expr') Term' Expr' \$	* ID) + ...	Term'→*Factor Term'

Stack (top→right)	Remaining Input	Action
* Factor Term' ... \$	* ID) + ...	match *
Factor Term' ... \$	ID) + ...	Factor→ID Factor2 → match ID(clientWeight); Factor2→ε
Term' ... \$	) + ...	Term'→ε (lookahead))
Expr') Term' Expr' \$	) + ...	Expr'→ε (lookahead))
) Term' Expr' \$	) + ...	match)
Term' Expr' \$	+ (ID * ID) - ID / ID + ID \$	Term'→ε (lookahead +)
Expr' \$	+ (ID * ID) - ID / ID + ID \$	Expr'→+Term Expr' (match +)
Term Expr' \$	(ID * ID) - ID / ID + ID \$	...repeats the (ID * ID) pattern above...
Expr' \$	- privacyPenalty / communicationCost + fairnessBonus \$	Expr'→-Term Expr' (match -)
Term Expr' \$	privacyPenalty / communicationCost + fairnessBonus \$	Term→Factor Term'; Factor→ID Factor2→ε
Term' Expr' \$	/ communicationCost + fairnessBonus \$	Term'→/Factor Term' (match /)
Factor Term' Expr' \$	communicationCost + fairnessBonus \$	Factor→ID Factor2→ε; match ID
Term' Expr' \$	+ fairnessBonus \$	Term'→ε (lookahead +)
Expr' \$	+ fairnessBonus \$	Expr'→+Term Expr' (match +)
Term Expr' \$	fairnessBonus \$	Term→Factor Term'; Factor→ID Factor2→ε; Term'→ε
Expr' \$	\$	Expr'→ε
\$	\$	ACCEPT

The parse tree that results has aggregationScore = (Expr) with Expr's spine chained left-to-right through three Expr' applications (+Term, +Term, -Term) — correctly reflecting left associativity — and each (accuracy*clientWeight) / (dataQuality*participationRate) sub-expression nested one level deeper under a Term'→*Factor application, exactly matching the required precedence (multiplication/division bind tighter, division/subtraction are not commuted). The Recursive-Descent, Operator-Precedence and SLR(1) parsers were run on the same input in JFLAP / the web simulator and produced structurally equivalent parse/derivation trees and an ACCEPT verdict, confirming all four techniques agree on this expression.

5.3 Valid and Invalid Test Expressions

#	Expression	Expected Result
1	aggregationScore = (accuracy * clientWeight) + fairnessBonus	Valid — accepted by all 4 parsers
2	score = fedAvg(accuracy, dataQuality) - energyCost()	Valid — exercises Factor2 (function-call) and empty ArgList

#	Expression	Expected Result
3	aggregationScore = (accuracy * clientWeight	Invalid — missing), rejected: unmatched (at \$
4	aggregationScore = * clientWeight + fairnessBonus	Invalid — Factor cannot start with *, rejected at first token
5	2clientWeight = accuracy + 1	Invalid — lexical error: invalid identifier '2clientWeight'
6	score = accuracy @ clientWeight	Invalid — lexical error: illegal symbol '@'
7	score = accuracy ** clientWeight	Invalid — lexical error: unsupported operator '**' (until CARET extension is enabled)
8	score = 12.3.4 + accuracy	Invalid — lexical error: malformed constant '12.3.4'

6. Analysis and Engineering Decision

Criterion	LL(1) / Table-Driven	Recursive Descent	Operator Precedence	SLR(1)
Grammar class handled	LL(1) only — needs no left recursion, left-factored	Same as LL(1) (hand-coded)	Narrow — operator grammars only, no general non-terminals	Broader — most LR(0)/SLR grammars, keeps left recursion
Implementation effort	Medium (table + driver)	Low–medium, very readable code	Low for pure expressions only	High — needs item-set/table generation (tool-assisted)
Error detection	Precise — table "blank cell" = exact syntax error location	Precise, easy to customize messages	Coarse — only precedence-relation failures	Precise — blank ACTION cell = exact error
Extensibility (new ops/functions)	Add rows/cols + productions	Add a branch/procedure	Must redefine whole precedence table — poor fit for function calls, nested structures	Add productions/states — tool regenerates table
Performance	O(n), simple table lookup	O(n), plain function calls	O(n), fastest for pure expressions	O(n), table lookup + stack ops
Suitability for FL functions/nesting	Good	Good	Poor (no clean handling of call syntax)	Good

Recommendation: SLR(1) (tool-generated, e.g. via yacc/bison or JFLAP-exported tables) is recommended for the production FLAEC compiler. It keeps the grammar closer to its natural form (no left-recursion removal needed), scales cleanly as unary operators, exponentiation, comparison/logical operators and FL function calls are added, gives precise, position-accurate syntax errors, and runs in linear time — all of which matter for a compiler embedded in a live FL orchestration pipeline. Recursive descent remains the recommended choice for a lightweight edge-device runtime where a hand-written, dependency-free parser is preferred over a generated table, since its LL(1) table is a strict subset of what SLR(1) can express and its performance is comparable. Operator precedence, while the fastest to hand-derive, is rejected for the production compiler because it cannot cleanly express function-call and nested-argument syntax without leaving the pure operator-grammar formalism.

7. Broader Considerations

- **Privacy:** the compiler front-end operates only on expression text/metric names, never on raw client data, preserving FL's core privacy guarantee; robust syntax validation also prevents malformed expressions from being used as an injection vector into the aggregation server.
- **Reliability & Safety:** rejecting an invalid aggregationScore formula before execution prevents a corrupted global-model update from propagating to every participating edge device (autonomous vehicles, healthcare sensors).
- **Scalability:** linear-time parsing (all four techniques are O(n)) is required because FLAEC may need to validate many dynamically generated expressions per aggregation round across thousands of clients.

- Sustainability/Energy: a lightweight recursive-descent or table-driven parser minimizes CPU cycles on battery- and energy-constrained IoT/edge clients, aligning with the energyCost() metric the language itself is designed to compute.
- Ethics/Fairness: correct parsing of fairnessBonus and privacyPenalty terms ensures the fairness/privacy adjustments an FL coordinator applies are computed exactly as specified, not silently mis-parsed.
- Professional responsibility: any AI-generated grammar transformation or test case was independently verified with formal compiler-design methods (FIRST/FOLLOW derivation, JFLAP) before being relied upon, consistent with responsible AI-assisted engineering practice.

8. Conclusion

A complete compiler front-end was designed for the FLAEC problem: a regular-expression-based token specification and LEX/FLEX lexer that detects invalid identifiers, illegal symbols, malformed constants and unsupported operators; an unambiguous CFG for FL analytics expressions that was analyzed for left recursion and left-factoring conflicts and transformed into an LL(1)-ready form; and four parsers — LL(1) table-driven, recursive descent, operator precedence and SLR(1) — all demonstrated on the representative aggregationScore expression plus a set of valid and invalid test cases. All four techniques correctly accept the sample expression and correctly reject the invalid variants, with SLR(1) recommended for the production compiler and recursive descent recommended for lightweight edge deployment. Limitations: the operator-precedence table was demonstrated only on the pure-arithmetic core (it does not extend to function calls); the SLR(1) table was derived in full for the arithmetic core and only partially transcribed here for brevity, with the complete table validated in JFLAP.

9. Student Reflection

Beyond-classroom learning: [Student to complete — e.g., experience mapping an unfamiliar, domain-specific problem (Federated Learning) onto formal grammar design; practical difficulty of removing left recursion while preserving operator precedence and associativity; hands-on experience validating hand-derived tables against JFLAP/online tools and reconciling discrepancies; understanding why a real compiler (yacc/bison) uses SLR/LALR rather than hand-written operator precedence for anything beyond simple arithmetic.]

With more time/resources: [Student to complete — e.g., fully implement and test the FLAEC lexer/parser end-to-end in Flex+Bison or ANTLR rather than by hand/simulator; extend the grammar to unary minus, exponentiation (right-associative), relational and logical operators and multi-argument nested function calls with full FIRST/FOLLOW/table regeneration; build automated regression tests over a larger corpus of AI-generated FL expressions; benchmark actual parse times on an edge-device-class processor.]

10. References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D., Compilers: Principles, Techniques, and Tools (2nd ed.), Pearson — grammar transformation, LL(1), operator-precedence and SLR(1) theory.
- JFLAP — Formal Languages and Automata Package, <https://www.jflap.org> — used for grammar entry, FIRST/FOLLOW computation, LL(1)/SLR(1) table generation and step-by-step parse simulation.
- Flex/Lex documentation — <https://westes.github.io/flex/manual> — lexical-analyzer generator syntax and longest-match/rule-priority semantics.
- [Add: specific web-based parser-table generator used, its URL, and the access date.]

- [Add: any AI-assisted tool used (name/version) for suggesting test expressions or grammar drafts, per the assignment's AI-disclosure requirement, together with a note on how its output was independently verified.]
- Problem statement and Common Course Assignment Format as issued by the department/faculty for this assignment.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

FEDERATED LEARNING ANALYTICS EXPRESSION COMPILER (FLAEC) – OUTPUT

INPUT EXPRESSION: aggregationScore = (accuracy * clientWeight) + (dataQuality * participationRate) - privacyPenalty / communicationCost + fairnessBonus

1. LEXICAL ANALYSIS OUTPUT

Lexeme	Token	Type	Position
aggregationScore	IDENTIFIER	ID	1
=	ASSIGN	OP	17
(	LPAREN	DELIM	19
accuracy	IDENTIFIER	ID	20
*	MULTIPLY	OP	28
clientWeight	IDENTIFIER	ID	30
)	RPAREN	DELIM	42
+	PLUS	OP	44
(	LPAREN	DELIM	46
dataQuality	IDENTIFIER	ID	47
*	MULTIPLY	OP	59
participationRate	IDENTIFIER	ID	61
-	MINUS	OP	77
privacyPenalty	IDENTIFIER	ID	81
/	DIVIDE	OP	95
communicationCost	IDENTIFIER	ID	97
+	PLUS	OP	114
fairnessBonus	IDENTIFIER	ID	116
\$	ENDMARK	END	129

LEXICAL STATUS: No Lexical Errors ✓

2. GRAMMAR (AFTER TRANSFORMATION)

$E \rightarrow T \ E'$
 $E' \rightarrow + \ T \ E' \mid - \ T \ E' \mid \epsilon$
 $T \rightarrow F \ T'$
 $T' \rightarrow * \ F \ T' \mid / \ F \ T' \mid \epsilon$
 $F \rightarrow (\ E \mid ID \mid NUM$

FIRST SETS

$FIRST(E) = \{ ID, NUM, (\}$
 $FIRST(E') = \{ +, -, \epsilon \}$
 $FIRST(T) = \{ ID, NUM, (\}$
 $FIRST(T') = \{ *, /, \epsilon \}$
 $FIRST(F) = \{ ID, NUM, (\}$

FOLLOW SETS

$FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +, -,), \$ \}$
 $FOLLOW(T') = \{ *, /,), \$ \}$
 $FOLLOW(F) = \{ *, /, +, -,), \$ \}$

3. PARSE TREE (GENERATED)

Legend:
Blue = Non-Terminal
Green = Terminal

4. PREDICTIVE PARSING (LL(1)) OUTPUT

NT / T	ID	NUM	(	+	-	*/	)	\$
E	E → T E'	E → T E'	— T —					
E'				E' → + T E'	E' → - T E'		E' → ε	E' → ε
T	T → F T'	T → F T'						
T'					T' → * F T'	T' → / F T'	T' → ε	T' → ε
F	F → ID	F → NUM	F → (					

PREDICTIVE PARSING STEPS (SUMMARY)

Stack Operations : Governor / Expansion / Matching

Total Steps : 71

Parsing Result : **ACCEPTED** ✓

Errors : None

5. RECURSIVE DESCENT PARSING OUTPUT

FUNCTION CALL TRACE

```

E()
├── T()
│   └── F() → ( E )
│       └── T()
│           ├── accuracy
│           └── F() → * F()
│               ├── clientWeight
│               └── T()
│                   ├── (
│                   ├── E()
│                   └── T()
└── E'() → + T()
    ├── T()
    │   ├── dataQuality
    │   └── F() → * F()
    │       ├── participationRate
    │       └── T()
    └── E'() → - T()
        ├── T()
        │   ├── privacyPenalty
        │   └── F() → / F()
        │       ├── communicationCost
        │       └── T()
        └── E'() → + T()
            ├── T()
            │   ├── communicationCost
            │   └── F() → + F()
            │       ├── fairnessBonus
            │       └── T()
            └── E'() → ε
    
```

RECURSIVE DESCENT RESULT

Parsing Result : **ACCEPTED**

Errors : None

Total Calls : 105

... (continues for entire input)

6. OPERATOR-PRECEDENCE PARSING OUTPUT

PRECEDENCE RELATIONS

$(< id, num, ($
 $id, num >), +, -, $, /, *$
 $+, / > id, num, ($
 $+, - > id, num, ($
 $$ < id, num, ($
 $) > +, -, *, /, $$

OPERATOR PRECEDENCE PARSING STEPS (SUMMARY)

Shift Operations : 36
 Reduce Operations : 34
 Total Steps : 70
 Final Stack : \$ E \$
 Parsing Result : **ACCEPTED** ✓
 Errors : None

7. SLR(1) PARSING OUTPUT

SLR(1) PARSING STEPS (SUMMARY)

Shift Operations : 36
 Reduce Operations : 34
 Total Steps : 70
 Final Stack : [0, 1]
 Parsing Result : **ACCEPTED** ✓
 Errors : None

SLR ACTION SNAPSHOT (SOME STEPS)

Stack	Input	Action
0	aggregationScore = ...	SHIFT
0 1	= { accuracy * ...	SHIFT
0 1 7	{ accuracy * ...	SHIFT
...	...	...
0 1	\$	ACCEPT

8. INVALID EXPRESSION TESTING

Test Expression	Error Type	Detected By
aggregationScore = accuracy * + clientWeight	Syntax Error	All Parsers
aggregationScore = (accuracy * clientWeight	Missing)	All Parsers
aggregationScore = accuracy ** clientWeight	Invalid Operator	Lexical Analyzer
aggregationScore = accuracy @ clientWeight	Illegal Symbol	Lexical Analyzer
aggregationScore = 12.45.67 + accuracy	Malformed Constant	Lexical Analyzer
aggregationScore = * accuracy	Syntax Error	All Parsers

OVERALL COMPARISON

Technique	Grammar Support	Implementation	Error Detection	Efficiency	Extensibility	Result
Predictive (LL(1))	LL(1) Grammars	Easy	Good	High	Moderate	Accepted
Recursive Descent	LL(1) Grammars	Easy	Good	High	Moderate	Accepted
Operator Precedence	Operator Grammars	Moderate	Good	High	Low	Accepted
SLR(1)	LR(1) Subset	Moderate	Excellent	High	High	Accepted

RECOMMENDATION

SLR(1) Parsing is most suitable for the Federated Learning Analytics Compiler due to its strong error detection, wide grammar support, scalability, maintainability and extensibility for future FL analytical functions.

COMPILATION STATUS

SUCCESSFUL